

KERNEL-LEVEL PACKET ENFORCEMENT ON PLUTOSDR

A deterministic Linux RX-handler firewall proof of concept for embedded SDR hardware

Author	Christopher T. Williams
Platform	PlutoSDR · ARMv7 hard-float · Linux 5.15.0-20952-ge14e351533f9-dirty
Document status	Engineering whitepaper · Proof of concept · October 2026
Scope	Kernel RX-path packet enforcement on usb0; separate from the P0 scanner

Executive summary

This paper documents a working proof of concept for selective packet enforcement on a PlutoSDR running embedded Linux. A loadable kernel module registers a receive handler on the USB network interface `usb0`, inspects the received `sk_buff`, and consumes packets matching a narrow IPv4 ICMP predicate. Non-matching traffic is returned to the normal receive path. In controlled tests recorded during development, ping to the Pluto management address failed while SSH remained available.

The principal engineering result is not merely packet classification: it is the demonstrated use of the kernel RX-handler boundary to stop selected packets from continuing through the ordinary receive path. The implementation is deliberately narrow and should be treated as a reproducible proof of concept, not a production-hardened firewall.

Key result

Observed: ICMP/ping was blocked by returning `RX_HANDLER_CONSUMED`; an SSH command returned `SSH_PASS` while the rule was active. A previous broad rule that consumed all IPv4 traffic interrupted SSH management, demonstrating why the classifier must remain narrow during testing.

Contents

1. Introduction and scope
2. Platform and constraints
3. Architecture and receive-path discovery
4. Deterministic classification and enforcement
5. Experimental method and results
6. Build and reproducibility

7. Safety, limitations, and threat model

8. Future work

Appendix A. Key implementation

Appendix B. Reproduction checklist

1. Introduction and scope

Embedded SDR platforms combine radio functionality with a general-purpose Linux networking stack. In this setting, a userspace packet observer can inspect copies of traffic without necessarily preventing the kernel from processing the original packet. Enforcement therefore requires a mechanism placed at a boundary where the kernel can decide whether the packet continues.

This project tests a small loadable kernel module that registers a network-device receive handler on `usb0`. Its current rule consumes IPv4 packets whose IP protocol field identifies ICMP. All other packets are passed onward. The firewall is a separate component from the P0 scanner; the scanner is outside this paper's implementation scope.

Contributions documented here are: (1) identifying the observed packet layout on the target receive path; (2) implementing a narrow ICMP match; (3) demonstrating selective packet consumption while preserving SSH; and (4) capturing enough kernel identity, configuration, source, and integrity metadata to support recreation on a matching device.

2. Platform and constraints

The tested device is a PlutoSDR based on ARMv7 hard-float, running Linux 5.15.0-20952-ge14e351533f9-dirty, build #10. The running configuration reports module support and symbol versioning (`CONFIG_MODULES=y`, `CONFIG_MODVERSIONS=y`). IPv6 and Netfilter were not enabled in the inspected kernel configuration. The deployed mechanism does not depend on iptables, Netfilter, tc, XDP, or a userspace firewall daemon.

The original module was built from the matching Pluto Linux source lineage. The recorded Linux source commit is `e14e351533f934047ba0473e836e561682ec67fe`; the firmware source lineage is `plutosdr-fw v0.38`. The build environment lacked the exact matching `Module.symvers`, so `modpost` emitted symbol-version warnings. Despite those warnings, the module loaded on the tested kernel and enforcement was observed. This is a reproducibility caveat, not a recommended release-build practice.

3. Architecture and receive-path discovery

The module registers `p0_rx_handler` on `usb0` using `netdev_rx_handler_register()`, under the RTNL lock. The handler receives a pointer to an `sk_buff`. During discovery, diagnostic logging of early packets showed that `skb->protocol` was IPv4 and that the first bytes at `skb->data` began with values such as `45 10 ...`, consistent with the IPv4 header beginning at the data pointer.

An initial classifier incorrectly assumed Ethernet-header offsets in `skb->data`. Under that assumption, the expected match did not occur and ping continued to succeed. The classifier was corrected to inspect the IPv4 header at the observed offset. This is an important platform-specific finding: packet offsets must be established on the actual receive path and not assumed from an unrelated capture path.

Logical packet flow

usb0 RX	<code>p0_rxhandler</code> registered RX handler	Classifier IPv4 + ICMP?
Incoming <code>sk_buff</code>	Inspect protocol and IPv4 header	Evaluate narrow predicate
Match	<code>RX_HANDLER_CONSUMED</code> stop normal receive processing	ICMP blocked
No match	<code>RX_HANDLER_PASS</code> continue receive path	SSH/TCP preserved in test

4. Deterministic classification and enforcement

The active predicate is intentionally minimal. It checks that the skb protocol is IPv4, that the version nibble at the start of the network header is 4, and that byte 9 of the IPv4 header—the Protocol field—equals IPPROTO_ICMP. The length check ensures at least 10 bytes are available before reading the Protocol field.

```
if (skb->len >= 10 &&
    skb->protocol == htons(ETH_P_IP) &&
    (skb->data[0] & 0xf0) == 0x40 &&
    skb->data[9] == IPPROTO_ICMP) {
    return RX_HANDLER_CONSUMED;
}
return RX_HANDLER_PASS;
```

The handler does not parse ICMP type or code, validate the IPv4 header length, or distinguish ICMP control messages. It matches the IPv4 protocol number broadly, so the current rule blocks IPv4 ICMP generally rather than only echo requests. The name “P0 signature” is a project label, not a claim that this predicate uniquely identifies malicious traffic.

5. Experimental method and results

Tests were conducted on the USB network path between FLEX and the Pluto management address. Before enforcement, ping to the target succeeded. With the narrow ICMP-consuming module loaded, a controlled ping test reported one transmitted packet and zero received packets (100% loss). An SSH test to the same management address returned SSH_PASS. These observations support selective enforcement for the tested path and rule.

Earlier diagnostic stages matter to the result. A first classifier used Ethernet offsets and did not match. A later test that consumed all IPv4 packets disrupted SSH management. After the classifier was narrowed to IPv4 ICMP, ping failed while SSH remained accessible. The broad-rule failure is a safety observation and is why the present module must not be generalized without explicit testing.

Evidence boundary: the recorded results are functional spot tests, not a benchmark or a comprehensive security evaluation. No throughput, latency, CPU, memory, packet-loss-under-load, race, or adversarial-header measurements are claimed.

6. Build and reproducibility

The recreation package includes the known-good `p0_rxhandler.ko`, the C source, Makefile, running kernel configuration, kernel identity, command line, README, and SHA256SUMS. The known-good module SHA-256 is:

```
9c85afaa9549ab03d7f3c49ddeae44fb00e23b47be964a47998fcd792c033a6
```

The source/build environment is maintained separately from the Pluto deployment archive. The original build used the ARM hard-float cross-toolchain and a matching Linux source tree. A recreation should first verify the exact kernel release and build identity, then build against the corresponding kernel source and configuration. Because `CONFIG_MODVERSIONS=y`, a production-quality rebuild should obtain the exact matching symbol-version data and verify module compatibility instead of relying on a build with missing `Module.symvers`.

```
LD_LIBRARY_PATH=/data/armhf-toolchain/usr/lib/x86_64-linux-gnu \
make ARCH=arm \
  CROSS_COMPILE=/data/armhf-toolchain/usr/bin/arm-linux-gnueabi- \
  EXTRA_CFLAGS="-march=armv7-a" \
  -C /data/pluto-linux \
  M=/data/pluto-linux/p0_rxhandler \
  modules
```

Deployment is manual. The module is not configured for automatic loading. On a matching test device, verify the kernel identity, verify the module hash, load with `insmod`, inspect kernel logs, run the ICMP and SSH tests, and remove with `rmmmod p0_rxhandler` when finished.

7. Safety, limitations, and threat model

This prototype is an interface-specific RX-handler proof of concept. Its behavior depends on kernel version, driver path, skb layout, and interface registration semantics. It is not a general policy engine and does not provide configurable rules, IPv6 handling, connection tracking, rate limits, counters suitable for production operations, persistent configuration, automatic rollback, or a remote recovery channel.

The current rule is a broad ICMP match, not a payload signature or a complete packet parser. The handler relies on the observed data layout and assumes the relevant bytes are accessible at `skb->data`. More defensive code should use kernel-supported header access patterns and account for skb linearity and header offsets if the implementation is extended to other paths or protocols.

A receive handler returning `RX_HANDLER_CONSUMED` prevents the packet from continuing through the ordinary receive path at that point. It should not be described as a universal filter for all ingress/egress traffic on the device: the tested registration is on `usb0` and the observations apply to that path. Other interfaces, locally generated traffic, alternate receive paths, and traffic that does not traverse this handler are outside the demonstrated scope.

The module uses GPL metadata in its compiled module declaration (`MODULE_LICENSE("GPL")`). The source and documentation bundle carries the custom permission and warranty notice supplied by the author. Those notices should be reviewed together before redistribution; the module's kernel license declaration is not, by itself, a substitute for a complete legal licensing analysis.

8. Future work

Priorities for engineering maturity are: (1) build against the exact kernel build artifacts and symbol-version data; (2) add a watchdog or out-of-band recovery path before testing broader rules; (3) add structured counters and configurable rules without weakening deterministic behavior; (4) test IPv4 header options, malformed and truncated packets, non-linear `sk_buffs`, fragments, and traffic on alternate interfaces; (5) measure throughput, latency, CPU load, and packet loss; (6) test lifecycle/error paths and repeated load/unload; and (7) document a reproducible automated test harness.

The P0 scanner may be evaluated as a separate module or process, but its resource footprint, build compatibility, and interaction with the firewall must be assessed independently. It is not included in the current firewall proof of concept.

Appendix A. Key implementation

The module registers the handler on `usb0` during initialization, while holding the RTNL lock. On successful registration it emits a kernel log message. During module removal it retrieves the device, unregisters the handler under RTNL, releases the device reference, and logs the packet count. Diagnostic logging is limited to the first 20 packets, including a 40-byte prefix when available.

Appendix B. Reproduction checklist

- Record ``uname -a``, ``/proc/version``, ``/proc/cmdline``, and ``/proc/config.gz``.
- Confirm target kernel release and build match the module target.
- Verify SHA256SUMS before loading.
- Load only on a recoverable test device with physical/replug access.
- Confirm ``p0_rxhandler``: registered on `usb0`` in kernel logs.
- Run a controlled ping test and record packet loss.
- Run an SSH command and confirm management remains available.
- Unload the module and verify handler removal.

- Record all differences in kernel, driver, compiler, configuration, and test path.

Author notice

Copyright (c) 2026 Christopher T. Williams. Permission is granted to use, copy, modify, and distribute this software for research, testing, and security-analysis purposes, provided that the copyright notice and permission notice are retained. The software is provided "AS IS", without warranty of any kind; the author disclaims liability as stated in the project license notice.

Document status: initial whitepaper draft prepared from the recorded implementation and tests. It should be updated with measured performance data, a final verified diagram, and a reproducible test log before external publication.